TEXTURE SEGMENTATION

1. Start with a small image

Suppose our grayscale image is:

```
   10  11  10  12  11  10
   11  10  12  11  10  12
   10  11  10  12  11  10
200 210 195 205 200 210
210 200 215 205 210 200
195 205 200 210 205 215
```

> **Remember:**
> Each number = one pixel
> *0* = black
> *255* = white

Here we intentionally created two different textures:

Top Part

```
10  11  10  12  11  10
11  10  12  11  10  12
10  11  10  12  11  10
```

Values are very close.
➡️ **Smooth texture**

Bottom Part

```
200 210 195 205 200
        210
210 200 215 205 210
        200
195 205 200 210 205
        215
```

Values vary more.
➡️ **Different texture / more variation**

2. We don't calculate texture from one pixel

Suppose our current pixel is:

```
          ↓
  10  11  10
11  [10]  12
  10  11  10
```

The current pixel is ***10***. But we don't say: *"10 means smooth."*

Instead, we look at the neighborhood around it. For example, using a ***3 × 3*** window:

```
10  11  10
11  10  12
10  11  10
```

There are: ***3 × 3 = 9*** pixels.

3. Step 1 — Calculate Mean

The mean is simply the average. Our 9 pixels are:

```
10, 11, 10, 11, 10, 12, 10, 11, 10
```

Add them:

$$10 + 11 + 10 + 11 + 10 + 12 + 10 + 11 + 10 = 95$$

There are 9 pixels. Therefore:

$$Mean = 95 / 9 = 10.56$$

So the local mean around our current pixel is: **10.56**

4. What does this mean?

It means: The average intensity of the pixels surrounding the current pixel is ***10.56***.

The current pixel itself was: **10**
But the local average is: **10.56**

> This is important because texture is calculated from the neighborhood, not only from the current pixel.

5. Now calculate variance

Variance asks: **How much are the pixels different from their mean?**

Our mean is: 10.56. Now take each pixel and compare it with 10.56.

Pixel 1 (Pixel = 10)
Difference: $10 - 10.56 = -0.56$
Square it: $(-0.56)^2 = 0.31$

Pixel 2 (Pixel = 11)
Difference: $11 - 10.56 = 0.44$
Square it: $(0.44)^2 = 0.19$

We do this for all 9 pixels:

Pixel	Difference from mean	Squared difference
10	-0.56	0.31
11	0.44	0.19
10	-0.56	0.31
11	0.44	0.19
10	-0.56	0.31
12	1.44	2.07
10	-0.56	0.31
11	0.44	0.19
10	-0.56	0.31

Add these squared differences:

$$0.31 + 0.19 + 0.31 + 0.19 + 0.31 + 2.07 + 0.31 + 0.19 + 0.31 \approx 4.19$$

Now divide by the number of pixels:

$$Variance = 4.19 / 9 \approx 0.47$$

So: **Variance ≈ 0.47**

6. What does variance = 0.47 tell us?

Very important. Our pixels were:

```
10  11  10
11  10  12
10  11  10
```

They are all very close to one another. Therefore:

```
LOW VARIANCE
     ↓
pixels are similar
     ↓
little local variation
     ↓
SMOOTH TEXTURE
```

7. Now take a rough region

Suppose our window is:

```
200  20  210
 30  195  40
220  25  205
```

These pixels are changing dramatically. Let's calculate the mean.

$$200 + 20 + 210 + 30 + 195 + 40 + 220 + 25 + 205 = 1145$$

There are 9 pixels. Therefore:

$$Mean = 1145 / 9 \approx 127.22$$

8. Now calculate variance

Look at the difference from the mean.

Pixel = **200**
$200 - 127.22 = 72.78$
Square: $72.78^2 \approx 5297$

Pixel = **20**
$20 - 127.22 = -107.22$
Square: $(-107.22)^2 \approx 11496$

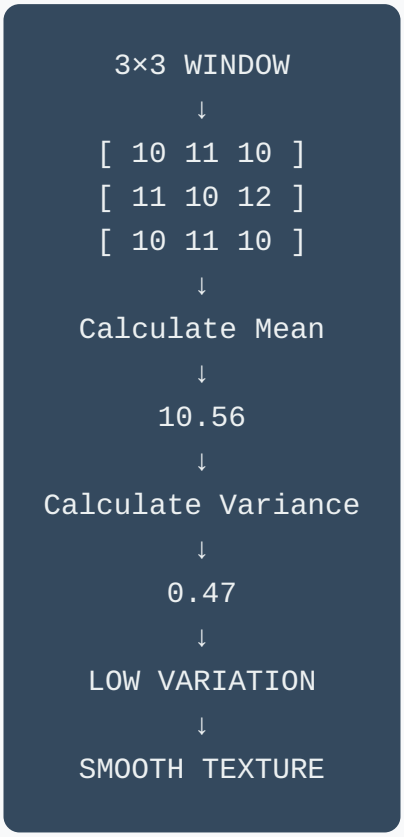
Already you can see the squared differences are very large. After doing this for all 9 pixels and averaging, the variance is very large compared with the previous example.

```
HIGH VARIANCE
      ↓
pixels are very different
      ↓
large local variation
      ↓
ROUGH / COMPLEX TEXTURE
```

9. So Mean and Variance have different jobs

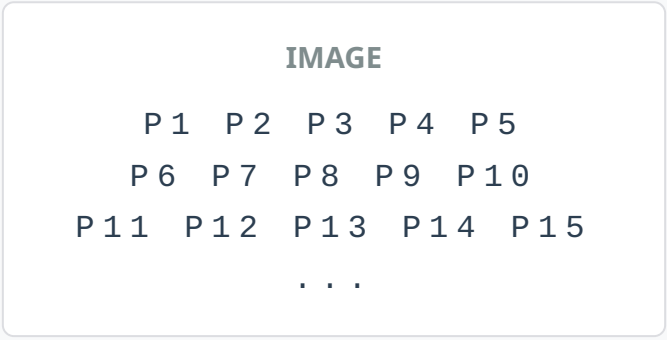
This is very important.

- **Mean tells us:** What is the average intensity in this neighborhood?
- **Variance tells us:** How much do the pixels vary around that average?

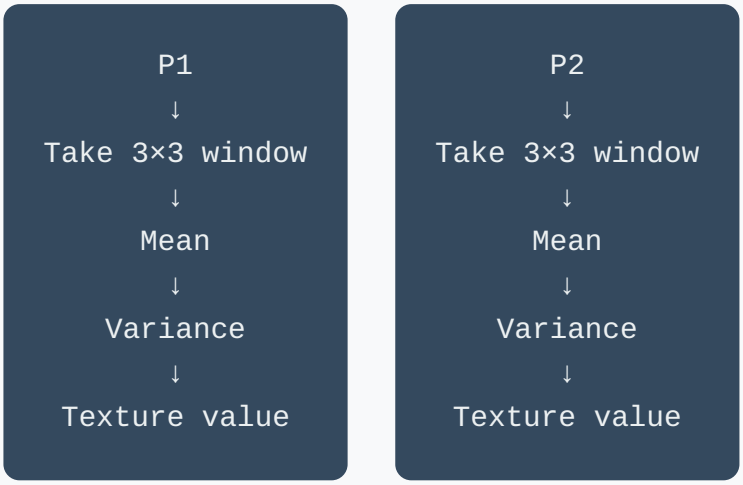


10. Now comes the most important part — moving the window

We don't calculate this only once. The window moves across the entire image.



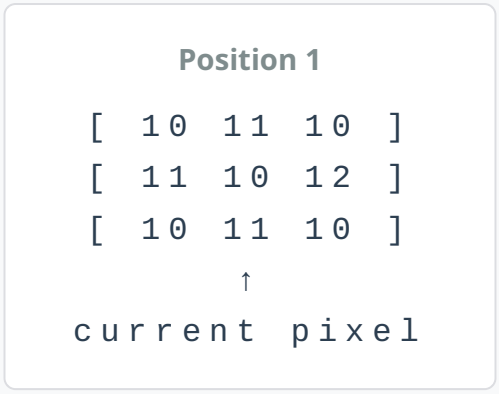
The algorithm does something like:

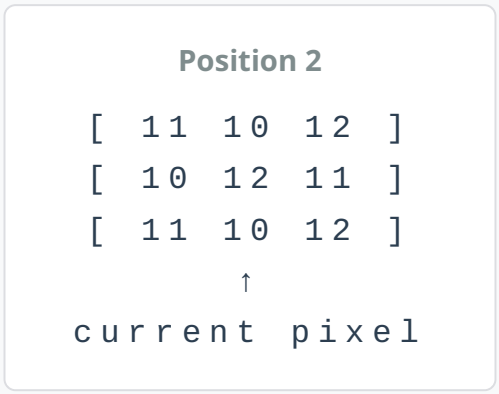


And continues across the image.

11. What exactly moves?

This is similar to Mean Shift. The image pixels don't move. Instead: **The window moves from one pixel location to another.**





```
Position 3

[  10  12  11  ]
[  12  11  10  ]
[  10  11  12  ]
        ↑
current pixel
```

For every position: WINDOW → MEAN → VARIANCE → TEXTURE VALUE

12. How does segmentation happen?

Suppose after moving the window over the image, we get Texture values:

```
   2   3   2   4   3
   3   2   4   3   2
   2   3   3   2   4

150 170 160 180 155
165 180 170 160 175
155 165 180 170 160
```

We can see two groups:

- **Group 1:** Values ~2-4. Low variance.
- **Group 2:** Values ~150-180. High variance.

Now we can create labels (0 = smooth, 1 = rough):

```
  0  0  0  0  0
  0  0  0  0  0
  0  0  0  0  0

  1  1  1  1  1
  1  1  1  1  1
  1  1  1  1  1
```

> **That is segmentation.**

13. But where does Mean come into segmentation?

Mean can also help distinguish textures. Both regions can have low variance, because both are internally consistent. But their mean intensities are different.

> **Region A:**
> Mean ≈ 10
> Variance ≈ low

> **Region B:**
> Mean ≈ 203
> Variance ≈ low

Therefore, using mean + variance together gives more information.

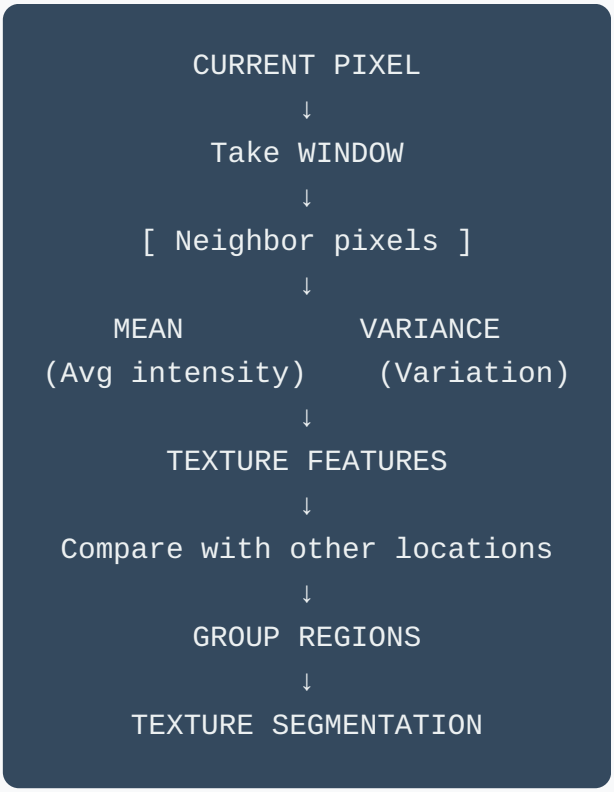
14. Why use both?

Imagine two windows. Both are smooth, but their brightness is different.

- Window A → (Mean = 10, Variance = low)
- Window B → (Mean = 100, Variance = low)

This gives the computer more information to differentiate them.

15. A very useful way to think about it



16. What happens at the next pixel?

As the window traverses the image:

- **P1:** Mean = 10.56, Variance = 0.47 → Smooth
- **P2:** Mean = 10.78, Variance = 0.60 → Smooth
- **P3:** Mean = 11.20, Variance = 0.80 → Smooth
- ...
- **P20:** Mean = 125, Variance = 5000 → Rough

So the algorithm can detect that it has entered another type of region.

17. One very important correction about "texture"

Don't think: *High variance = always one specific texture.*

Variance only tells us local intensity variation. For example: **Grass, Brick, Hair, Fabric** can all have high variation.

To identify different textures more reliably, we calculate more advanced features:

- Local variance & Standard deviation
- Contrast & Entropy

- GLCM features
- Local Binary Pattern (LBP)
- Gabor filter responses

But for understanding the basic principle, mean + variance is excellent.

18. Connect this with the code

The OpenCV code conceptually performs these operations:

```
mean = cv2.blur(img, (5, 5))
```

For every pixel, calculate the average of the surrounding 5×5 pixels.

```
mean_square = cv2.blur(img * img, (5, 5))
```

Average of the squared pixel values in the 5×5 neighborhood.

```
variance = mean_square - (mean * mean)
```

Implements: $Variance = Mean(X^2) - [Mean(X)]^2$. Therefore, for every pixel, we obtain a local variance.

19. Final complete picture

> **The easiest memory formula:**
>
> **Mean** = "What is the average here?"
> **Variance** = "How much are the pixels changing here?"
> **Texture segmentation** = "Calculate these local properties around every pixel and use them to separate regions with different local patterns."


IMAGE
↓
Start at first pixel
↓
Put 3×3 / 5×5 window
↓
Read all pixels inside
↓
Calculate MEAN and VARIANCE
↓
Describe local texture
↓
Move window → next pixel
↓
Repeat until every pixel gets texture information
↓
Compare texture features
↓
Similar texture = same region
↓
DIFFERENT REGIONS